© Guven

Transcription with a Transformer

```
"Attention is all you need." Vaswani
```

A Transformer is designed to process sequential data and map input sequences to output sequences, solving tasks such as language translation and text summarization.

Let's build a model that can transcribe signals to letters. For example, the following code trains a Transformer model with general attention to classify sine wave signals into corresponding musical note letters. It is, in effect, a sequence-to-sequence model. Note that the model looks like a classifier, but because of its sequence-to-sequence nature, it can also be used as a generator. For example, it can be architected to learn letters and generate signals accordingly, mapping letters to signals.

🔶 Dataset

`Note_Freqs` is a dictionary mapping note names (e.g., 'C4', 'A4') to their frequencies, so these frequencies can be used to generate the actual sine wave by the following equation,

$$\text{sine_wave} = \sin\left(2\pi f \frac{x}{1000}\right) \tag{1}$$

- Since the sampling frequency is 1kHz (i.e., the denominator $1/1000$), each time slot t is 1 millisecond
- `self.data` is a tensor of sine waves, shape: `n_samples` $\times$ `seq_len`
- `self.labels` is a tensor of note labels, shape: `n_samples`

Output can be iterated, and the dataset returns pairs `(sine_wave, label)` via a `DataLoader`, for example.

🔶 Model

The `TransformerNotePredictor` class is based on a Transformer model. The input is a sine wave signal tensor of shape `seq_len`. The model has,

- Embedding layer, which projects input sine wave values into a higher-dimensional space

- Positional encoding, which adds trainable positional information to the embeddings as an additive component
- Transformer encoder, `Transformer` layer from PyTorch
  - Processes the sequence to capture temporal dependencies
  - The encoder expects input shape `seq_len` × `batch_size` × `model_dim` so the input is permuted accordingly
- Classification head
  - Averages the encoder output over the sequence length
  - Applies a linear layer to map to the number of note classes
  - Outputs logits for each note class

The output is logits tensor of shape `batch_size` × `n_classes` representing the predicted note probabilities

In [2]:
```
%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 72
import numpy as np

import torch
import torchaudio
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader

Device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

print(f'PyTorch version= {torch.__version__}')
print(f'torchaudio version= {torchaudio.__version__}')
print(f'CUDA available= {torch.cuda.is_available()}')
```

```
PyTorch version= 2.9.1+cu130
torchaudio version= 2.9.1+cpu
CUDA available= True
```

In [3]:
```
# notes
Note_Freqs = {
 'C2': 65.41, 'C#2': 69.3, 'D2': 73.42, 'D#2': 77.78, 'E2': 82.41, 'F2': 8
 'F#2': 92.5, 'G2': 98.0, 'G#2': 103.83, 'A2': 110.0, 'A#2': 116.54, 'B2':
 'C3': 130.81, 'C#3': 138.59, 'D3': 146.83, 'D#3': 155.56, 'E3': 164.81, '
 'F#3': 185.0, 'G3': 196.0, 'G#3': 207.65, 'A3': 220.0, 'A#3': 233.08, 'B3
 'C4': 261.63, 'C#4': 277.18, 'D4': 293.66, 'D#4': 311.13, 'E4': 329.63, '
 'F#4': 369.99, 'G4': 392.0, 'G#4': 415.3, 'A4': 440.0, 'A#4': 466.16, 'B4
 'C5': 523.25, 'C#5': 554.37, 'D5': 587.33, 'D#5': 622.25, 'E5': 659.25, '
 'F#5': 739.99, 'G5': 783.99, 'G#5': 830.61, 'A5': 880.0, 'A#5': 932.33, '
 'C6': 1046.5, 'C#6': 1108.73, 'D6': 1174.66, 'D#6': 1244.51, 'E6': 1318.5
 'F#6': 1479.98, 'G6': 1567.98, 'G#6': 1661.22, 'A6': 1760.0, 'A#6': 1864.
 'C7': 2093.0, 'C#7': 2217.46, 'D7': 2349.32, 'D#7': 2489.02, 'E7': 2637.0
 'F#7': 2959.96, 'G7': 3135.96, 'G#7': 3322.44, 'A7': 3520.0, 'A#7': 3729.
}
Notes = list(Note_Freqs.keys())  # labels - notes
```

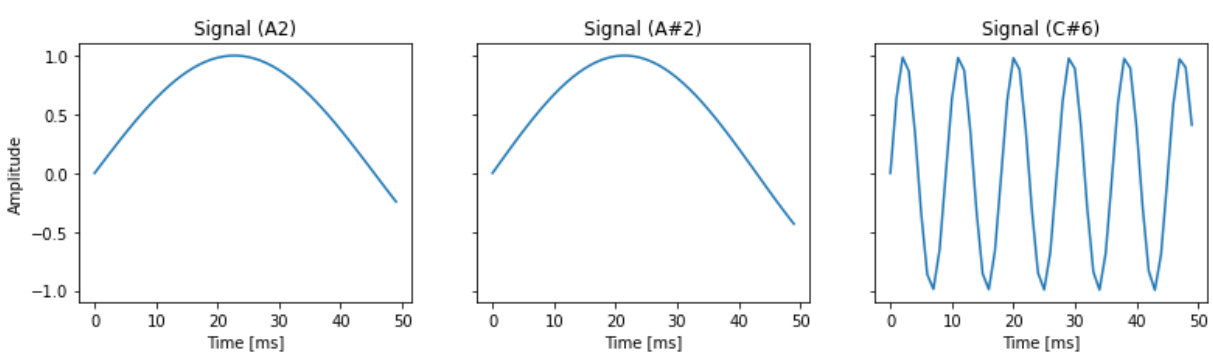
```python
SEQ_LEN= 50

# generate sinewaves for notes using the frequency in Note_Freqs
class NoteDataset(Dataset):
    def __init__(self, n_samples=512, seq_len=SEQ_LEN):
        self.data, self.labels = [], []
        for _ in range(n_samples):
            note = np.random.choice(Notes)  # randomly generate signals
            f = Note_Freqs[note]
            x = np.arange(seq_len)
            sine_wave = np.sin(2 * np.pi * f * x / 10000)  # sample at 10k
            self.data.append(torch.tensor(sine_wave, dtype=torch.float32))
            self.labels.append(Notes.index(note))  # encode note as index,

        self.data = torch.stack(self.data)
        self.labels = torch.tensor(self.labels, dtype=torch.long)  # label

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]

class TransformerNotePredictor(nn.Module):
    def __init__(self, input_dim=1, model_dim=16, nhead=2, n_encoder_layer
        super().__init__()
        self.embedding = nn.Linear(input_dim, model_dim)
        self.pos_encoding = nn.Parameter(torch.rand(SEQ_LEN, model_dim))
        self.transformer = nn.Transformer(model_dim, nhead, n_encoder_laye
        self.classifier = nn.Linear(model_dim, n_classes)  # output layer

    def forward(self, _x):
        # add positional encoding
        _x = self.embedding(_x.unsqueeze(-1)) + self.pos_encoding  # shape
        _x = _x.permute(1, 0, 2)  # Transformer expects (seq_len, batch, m
        _x = self.transformer.encoder(_x)
        _x = _x.mean(dim=0)  # average over seq len
        return self.classifier(_x)  # logits for classification

def train(_model, _dataloader, epochs=10, lr=0.001, info=False):
    import sys
    optimizer = optim.Adam(_model.parameters(), lr=lr)
    criterion = nn.CrossEntropyLoss()

    for e in range(epochs):
        totloss = 0
        for x, y in _dataloader:
            optimizer.zero_grad()
            out = _model(x)
            loss = criterion(out, y)
            loss.backward()
            optimizer.step()
            totloss += loss.item()

        if info:
```

```
            sys.stderr.write(f"\r{e+1:3d}/{epochs:3d} | Loss: {totloss:3.3
            sys.stderr.flush()


# generation function, given a sine wave predict the note
def predict(_model, _wav):
    _model.eval()
    with torch.no_grad():
        out = _model(_wav.unsqueeze(0))  # add batch dimension
        pred_note = Notes[out.argmax(dim=1).item()]
    return pred_note

# dataset
ds_notes = NoteDataset(n_samples=1024)  # there will be repeats
Dloader = DataLoader(ds_notes, batch_size=32, shuffle=False)
```

In [4]:
```
ds_notes[0][0]
```

Out[4]:
```
tensor([ 0.0000,  0.0691,  0.1378,  0.2059,  0.2730,  0.3387,  0.4029,  0.4
652,
         0.5252,  0.5827,  0.6374,  0.6891,  0.7375,  0.7824,  0.8235,  0.8
607,
         0.8938,  0.9227,  0.9471,  0.9670,  0.9823,  0.9929,  0.9987,  0.9
998,
         0.9961,  0.9877,  0.9745,  0.9567,  0.9343,  0.9075,  0.8763,  0.8
409,
         0.8016,  0.7584,  0.7115,  0.6613,  0.6079,  0.5516,  0.4927,  0.4
315,
         0.3681,  0.3030,  0.2365,  0.1688,  0.1004,  0.0314, -0.0377, -0.1
066,
        -0.1750, -0.2426])
```

In [5]:
```
# plot some signals
fig, ax = plt.subplots(1, 3, figsize=(13,3), sharey=True, dpi=72)
ax[0].plot(ds_notes[0][0])
ax[1].plot(ds_notes[1][0])
ax[2].plot(ds_notes[2][0])
ax[0].set(xlabel='Time [ms]', ylabel='Amplitude', title=f'Signal ({Notes[d
ax[1].set(xlabel='Time [ms]', title=f'Signal ({Notes[ds_notes[1][1]]})')
ax[2].set(xlabel='Time [ms]', title=f'Signal ({Notes[ds_notes[2][1]]})')
plt.show()
```



In [6]:
```
Model = TransformerNotePredictor()
```

In [7]:
```
%%time

train(Model, Dloader, epochs=100, info=True)
```

```
100/100 | Loss: 9.71734
CPU times: total: 48min 6s
Wall time: 2min 1s
```

In [8]:
```
# evaluation
print(f"Pred   Orig")
for _ in np.random.randint(0, len(ds_notes), size=20):
    d, t = ds_notes[_]
    pred = predict(Model, d)
    print(f"{pred:<4s}   {Notes[t]:<4s}")
```

```
Pred   Orig
B6     B6
B2     B2
C5     C5
E6     E6
C5     C5
D3     D3
C#7    C#7
B3     B3
G#2    G#2
F#5    F#5
B6     B6
F6     F6
F#4    F#4
D2     D#2
D#6    D#6
A#3    A#3
B4     B4
B4     B4
A6     A6
B2     B2
```

In [9]:
```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```